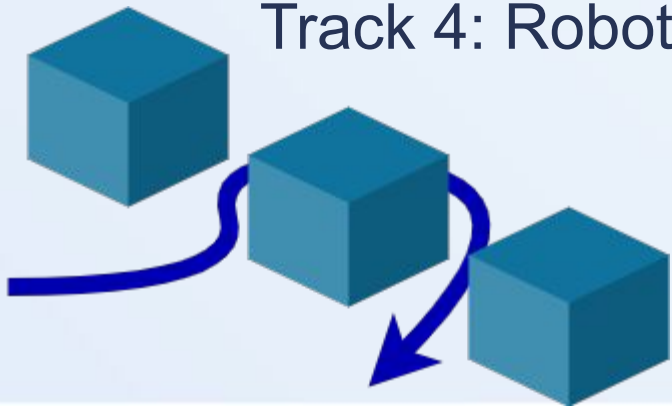


RRT Algorithm

RRT turns randomness into results.

We are Team 2

Track 4: Robotic Motion Planning. **How to move?**



Rapidly Exploring Random Tree (RRT) Algorithm

The RRT is a sampling-based motion planner that applies to robots with arbitrary dynamics of the form:

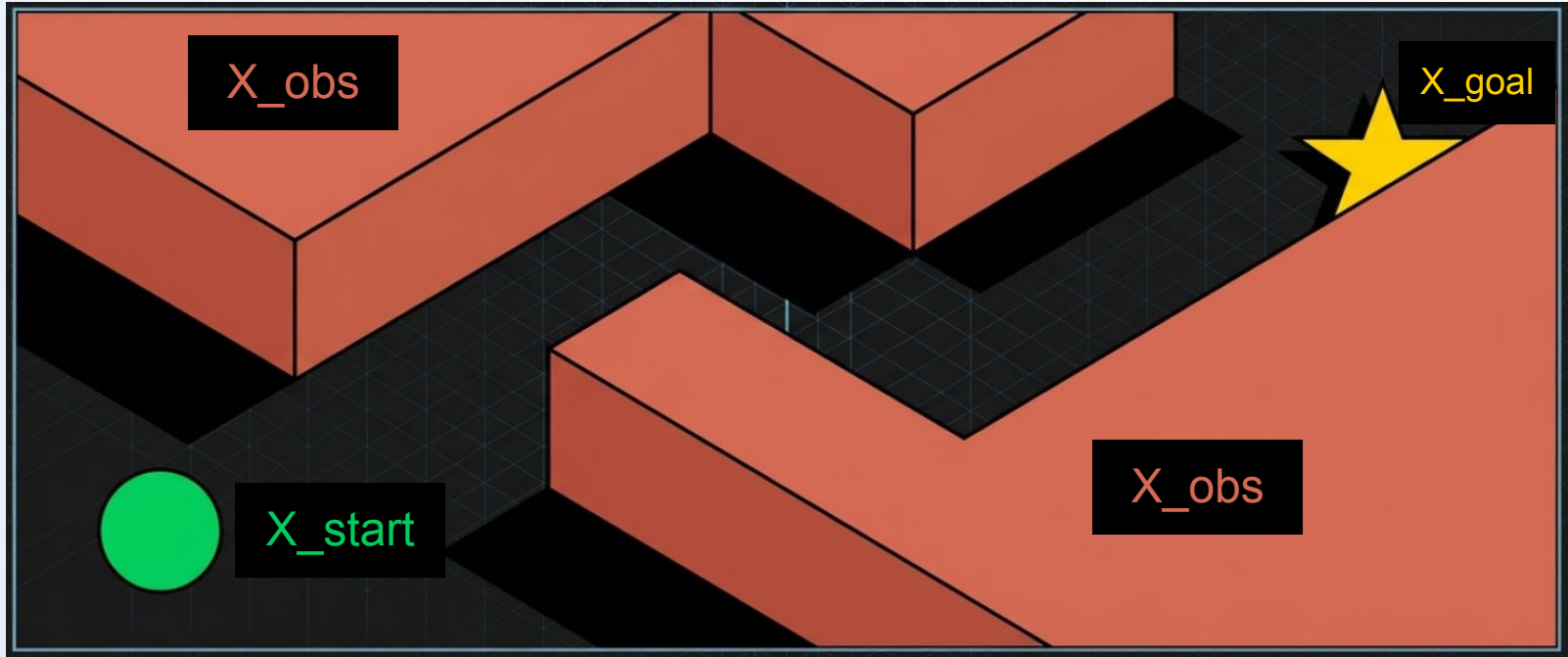
$$\dot{x}(t) = f(x(t), u(t))$$

Where,

- $x(t)$: State vector at time t
- $\dot{X}(t)$: Time derivative of the state.
- $u(t)$: Control input at time t .
- $f(\cdot)$: System dynamics function

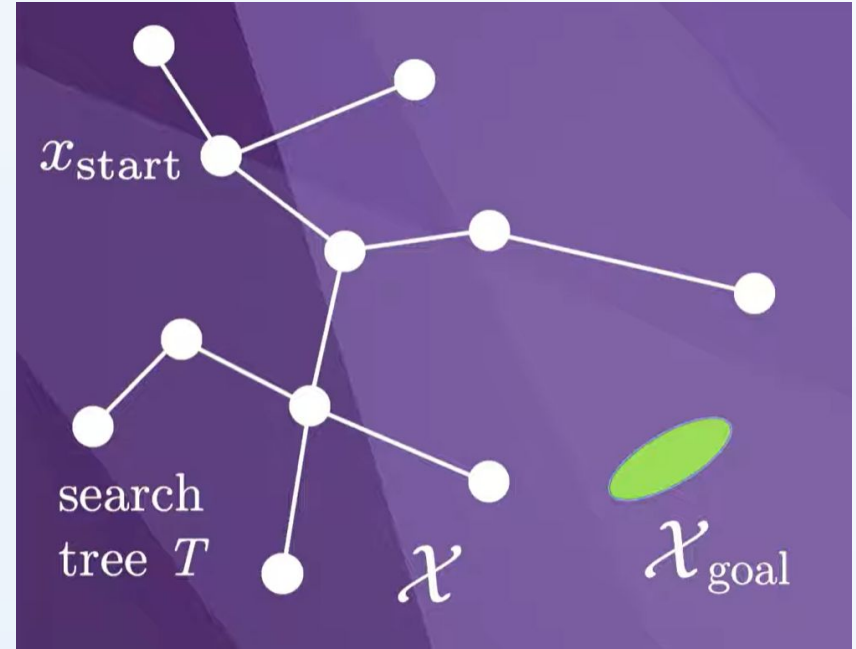
The Mission

Objective: Navigate X (Space) to reach X_{goal} avoid X_{obs} (Obstacles).



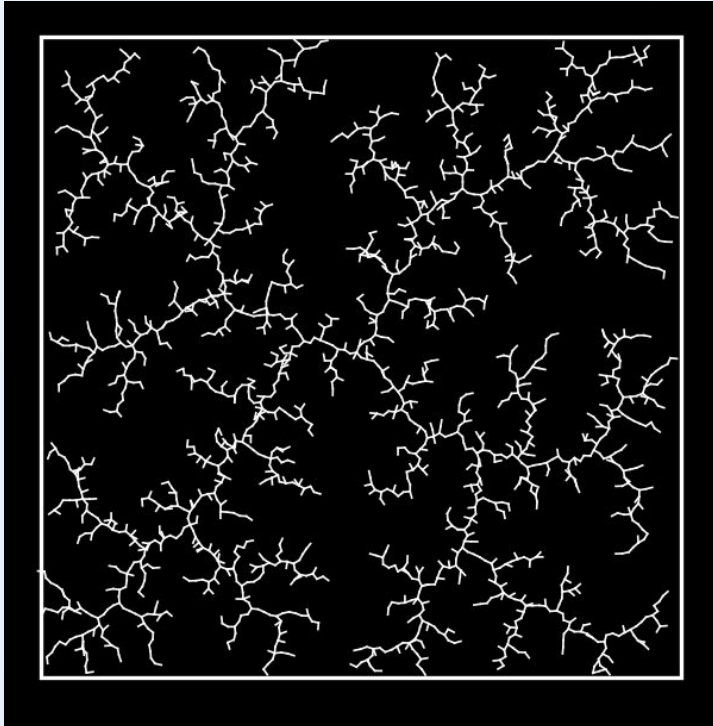
The Mission

Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.

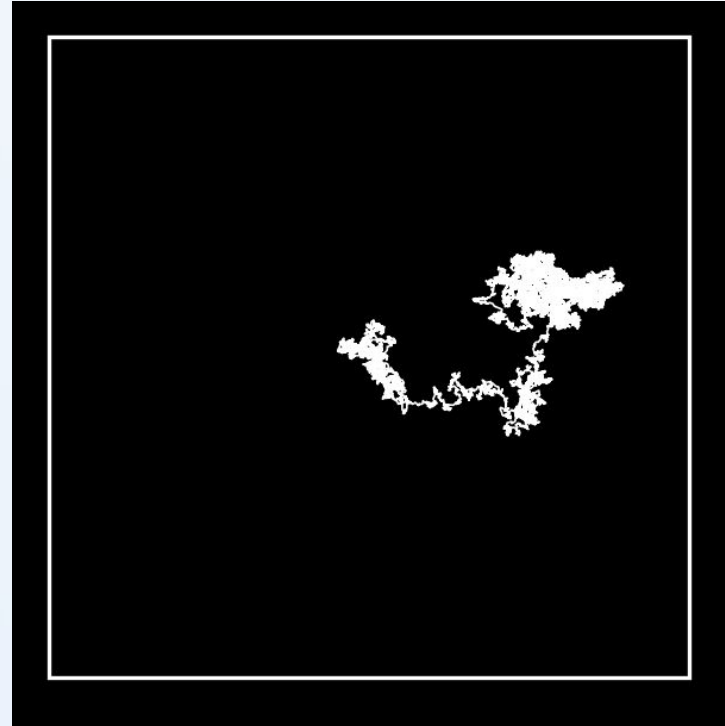


Comparison between RRT and Random Walk

RRT



Random Walk



Pseudocode of RRT Algorithm (1/2)

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Pseudocode of RRT Algorithm (1/2)

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Pseudocode of RRT Algorithm (1/2)

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Pseudocode of RRT Algorithm (1/2)

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Pseudocode of RRT Algorithm (1/2)

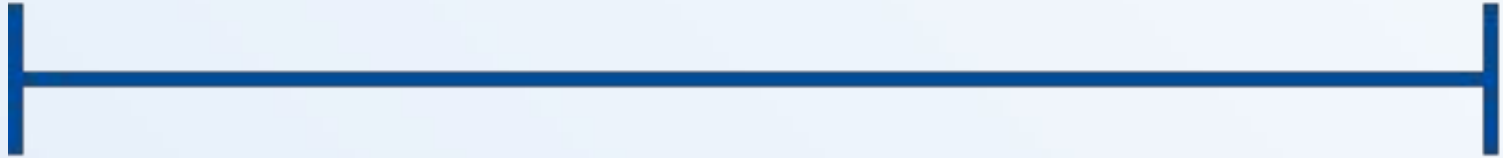
```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Pseudocode of RRT Algorithm (2/2)

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Van der Corput sampling from a 1D interval

Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.



Van der Corput sampling from a 1D interval

Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.



Van der Corput sampling from a 1D interval

Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.



Van der Corput sampling from a 1D interval

Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.



Van der Corput sampling from a 1D interval

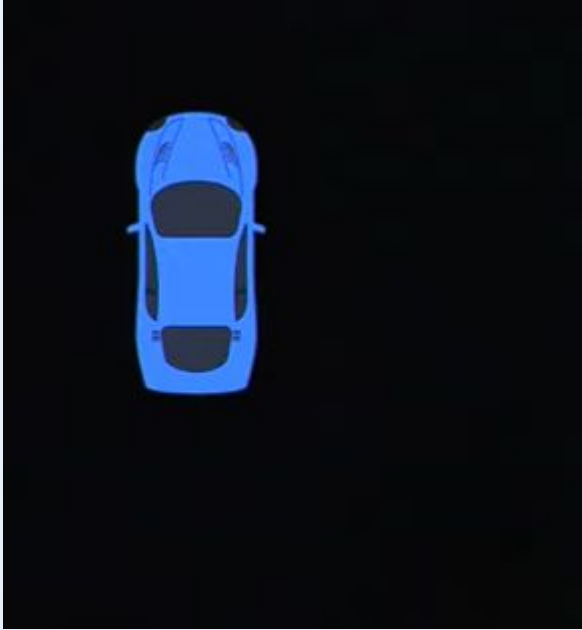
Every so often the sample X_{sample} should be chosen inside of X_{goal} to try to complete the planning problem.



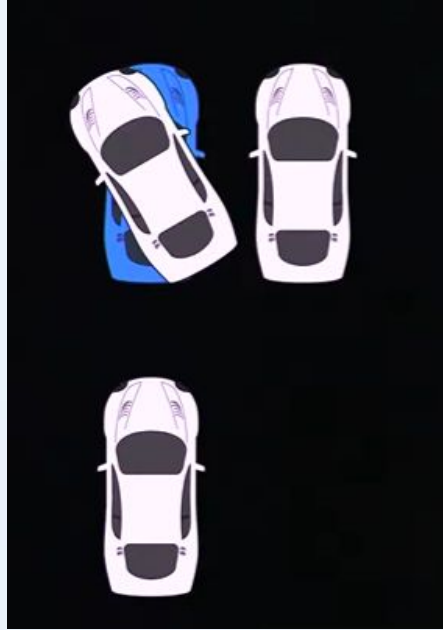
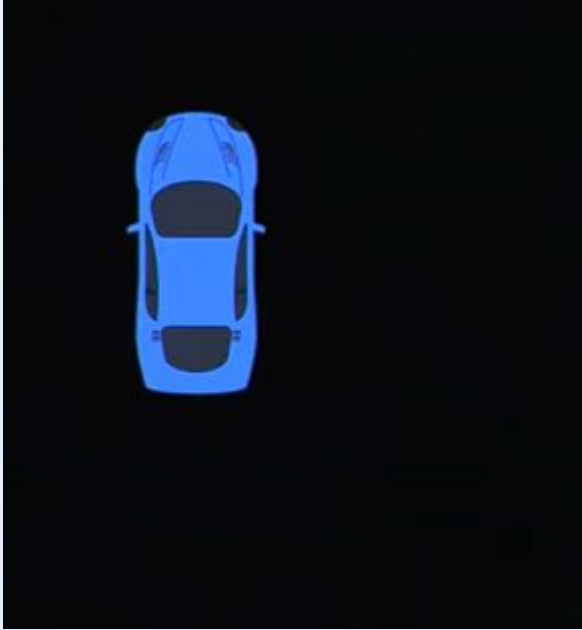
Pseudocode of RRT Algorithm

```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:     X_samp <- sample from X_samp
4:     X_nearest <- nearest node in T to X_samp
5:     employ a local planner to find a motion from X_nearest to X_new
        |   |   the direction of X_samp
6:     if the motion is collision-free then
7:         add X_new to T with an edge from X_nearest to X_new
8:         if X_new is in X_goal then
9:             return SUCCESS and the motion to X_new
10:        end if
11:    end if
12: end while
13: return FAILURE
```

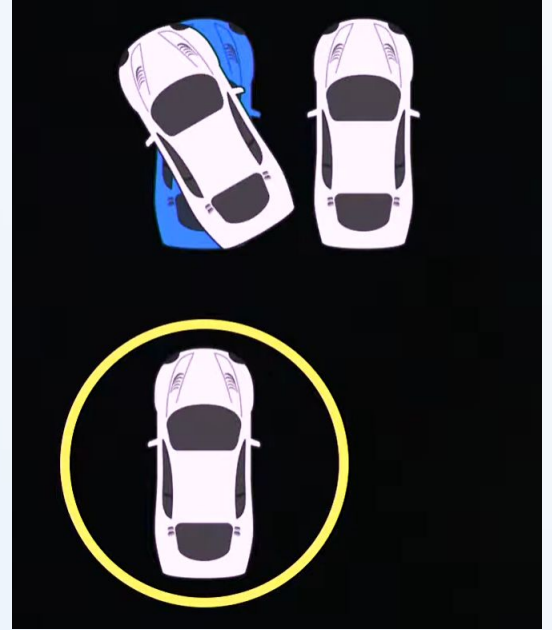
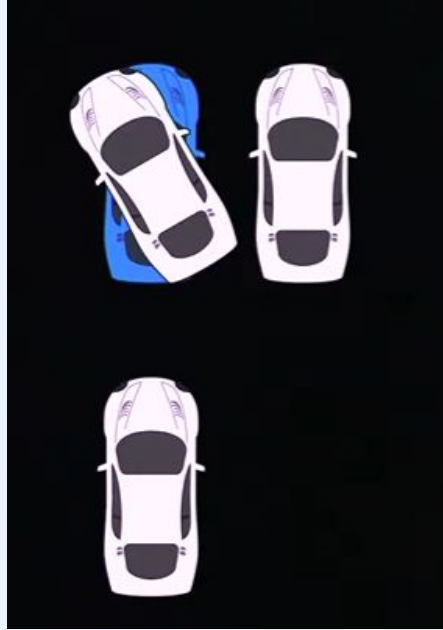
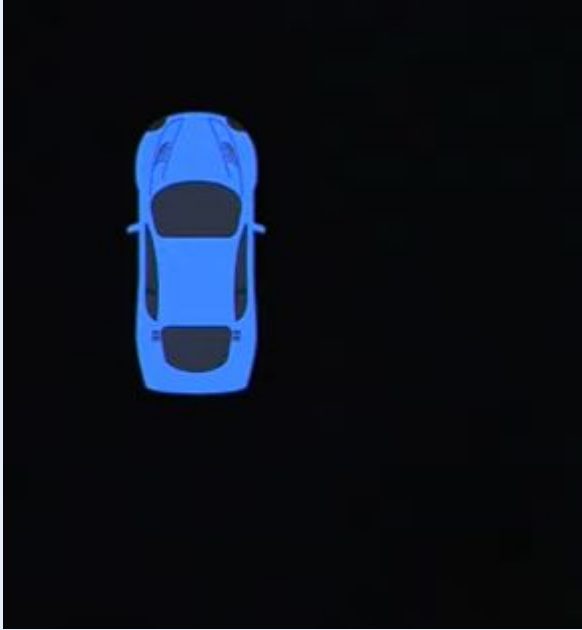
Why is Motion Planning Important?



Why is Motion Planning Important?



Why is Motion Planning Important?



Pseudocode of RRT Algorithm

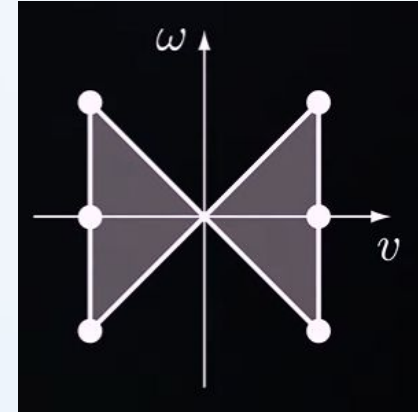
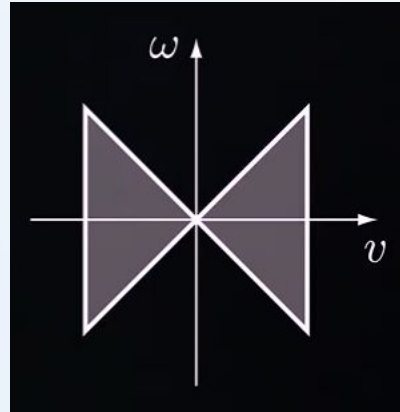
```
1: initialize search tree T with X_start
2: while T is less than the maximum tree size do
3:   X_samp <- sample from X_samp
4:   X_nearest <- nearest node in T to X_samp
5:   employ a local planner to find a motion from X_nearest to X_new
   |   |   the direction of X_samp
6:   if the motion is collision-free then
7:     add X_new to T with an edge from X_nearest to X_new
8:     if X_new is in X_goal then
9:       return SUCCESS and the motion to X_new
10:    end if
11:  end if
12: end while
13: return FAILURE
```

Examples of Local Planners

- Straight-line path for kinematic, fully actuated robots
- Integrated discretized control that gets “closest”

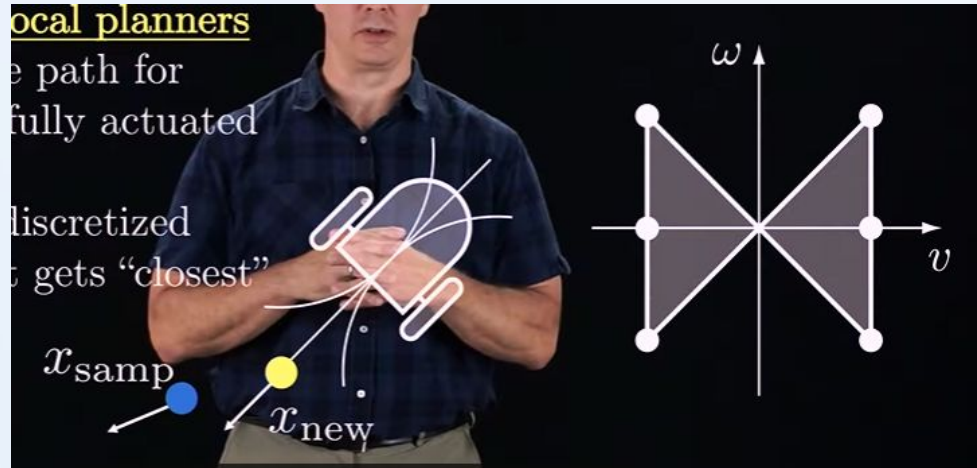
Examples of Local Planners

- Straight-line path for kinematic, fully actuated robots
- Integrated discretized control that gets “closest”



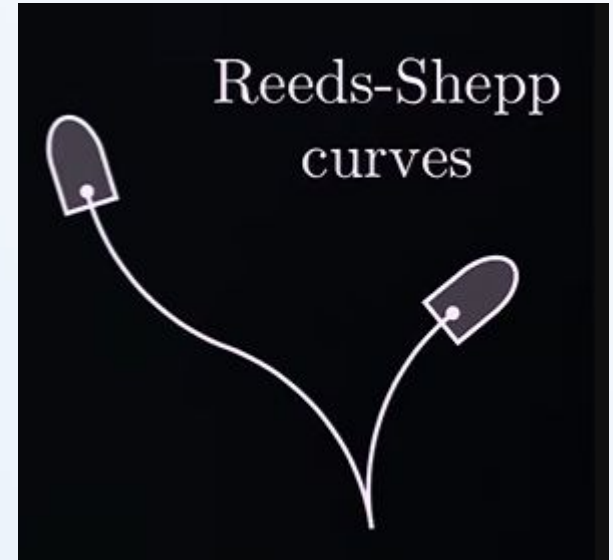
Examples of Local Planners

- Straight-line path for kinematic, fully actuated robots
- Integrated discretized control that gets “closest”



Examples of Local Planners

- Straight-line path for kinematic, fully actuated robots
- Integrated discretized control that gets “closest”
- Planner tailored to robot

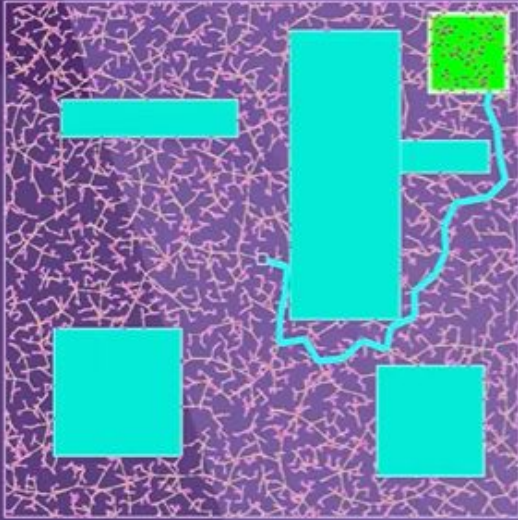


Examples of Local Planners

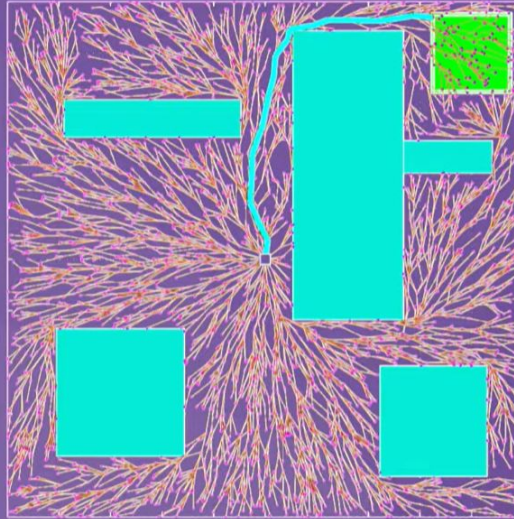
- Straight-line path for kinematic, fully actuated robots
- Integrated discretized control that gets “closest”
- Planner tailored to robot

Improvement: RRT*

RRT



RRT*

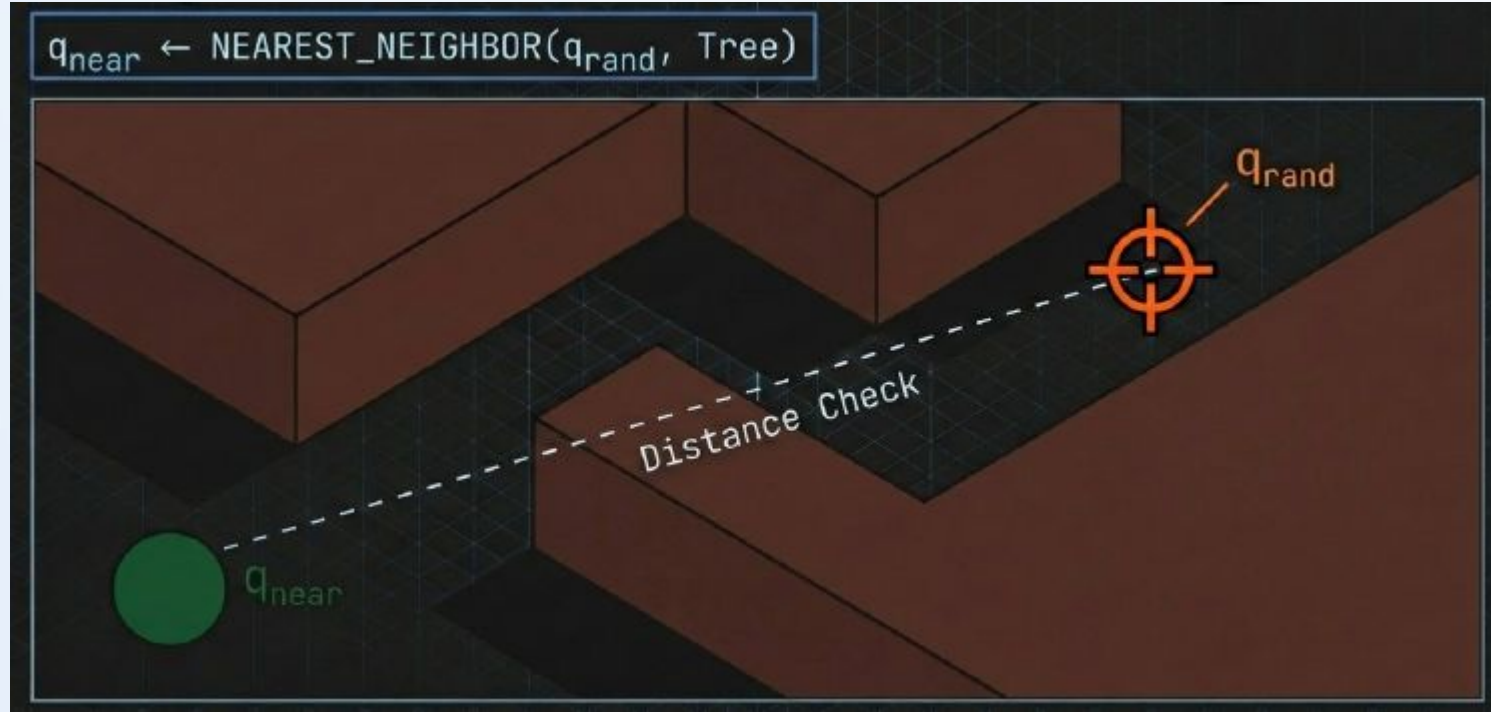


Karaman and
Frazzoli, *RSS* 2010

Recap: Step 1: Roll The Dice (Sampling)



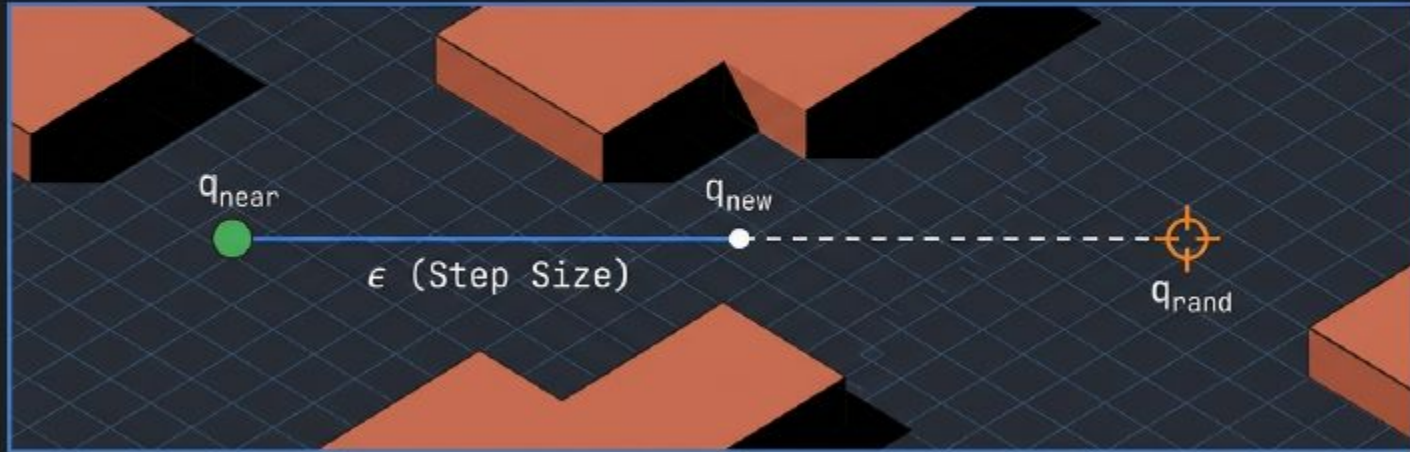
Recap: Step 2: Find the Closest Buddy



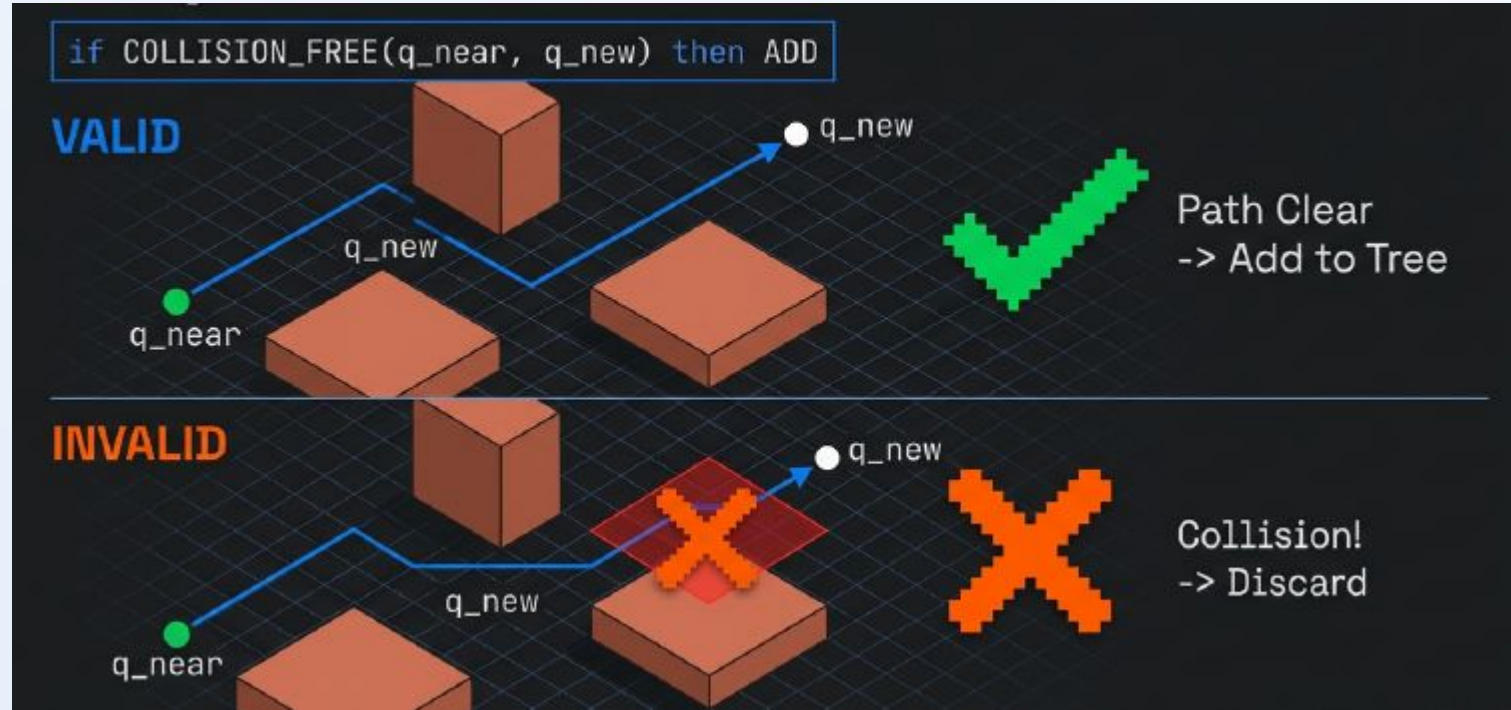
Recap: Step 3: Don't Jump, Just Step

```
 $q_{\text{new}} \leftarrow \text{STEER}(q_{\text{near}}, q_{\text{rand}}, \text{step\_size})$ 
```

Move fixed distance
toward target.

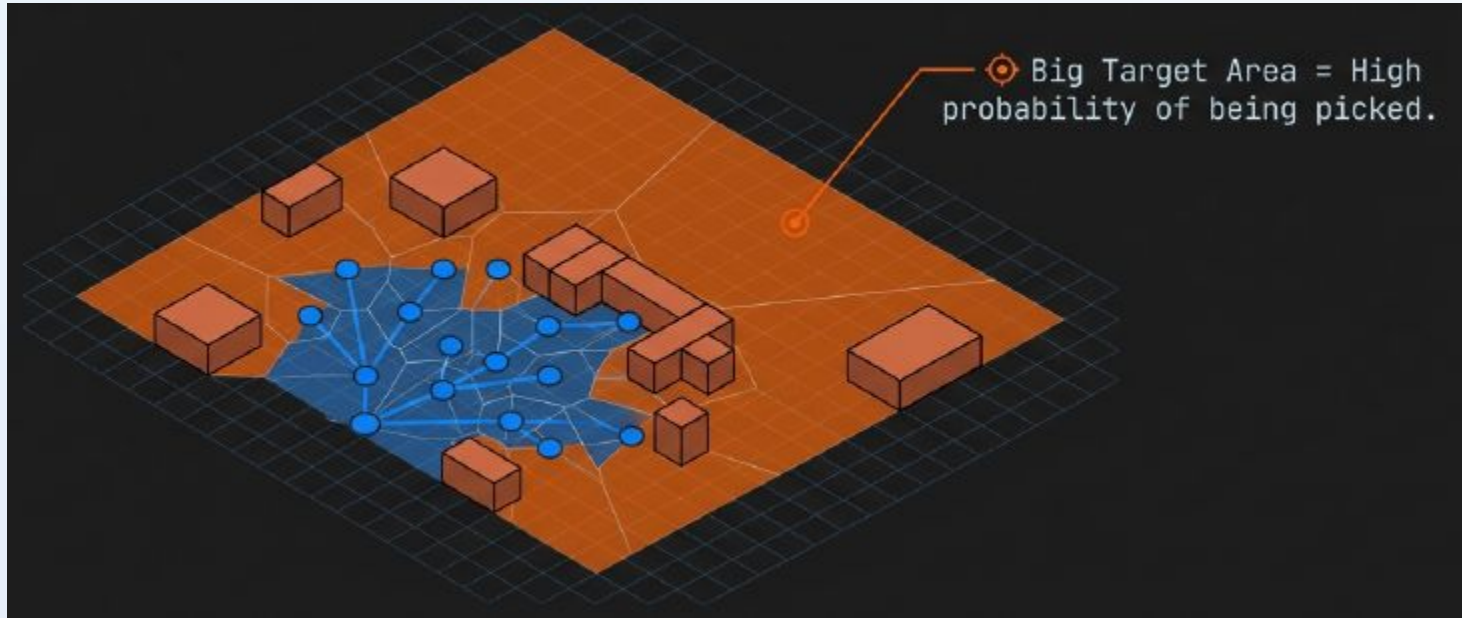


Recap: Step 4: Watch Your Head



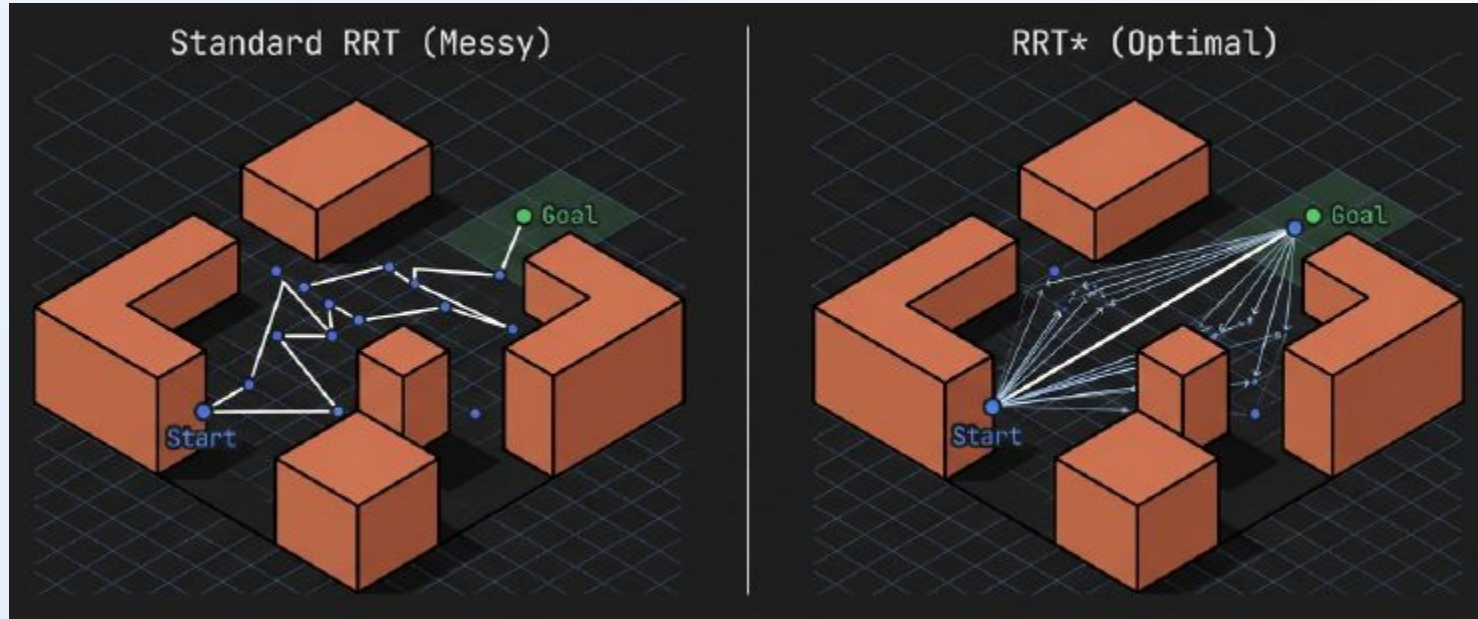
Why It Works: The Voronoi Bias

Large empty spaces = Large target probability



Level Up: RRT* (The Optimizer)

Adding a 'Rewire' step to find the shortest path.



A Live Visualization of the RRT Algorithm

Adding a 'Rewire' step to find the shortest path.